

Sum of Array

Given an array of N integer, design a recursive algorithm to compute the **sum** of array elements.

Example

Input

0	1	2	3	4
10	20	30	40	50

Output : 150

```
int s = 0;
for (i = 0; i < n; i++) {
    s += arr[i];
}
return s;
```

time: $O(n)$
space: $O(1)$

find the sum ($x_0 \dots x_{n-1}$)

$$x[] = \{x_0, x_1, x_2, \dots, x_{n-1}\}$$

$$S = x_0 + x_1 + x_2 + \dots + x_{n-1}$$

$\Downarrow$
 $S = x_0 + A$ (friend)

$\underbrace{\text{sum}(x[1 \dots n-1])}$
 suffix that starts at the 1st idx

$\Rightarrow f(i)$ is a fn that finds the sum ($x[i \dots n-1]$)

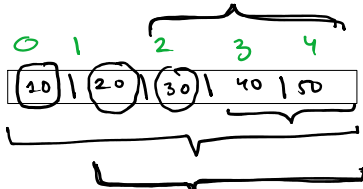
$$= x[i] + f(i+1)$$

$\Downarrow$
 it is a fn that finds the sum of $x[i+1 \dots n-1]$

$\underbrace{\text{sum}(x[i+1 \dots n-1])}$

init $i=0$

$f(0)$



$$f(0) = x[0] + f(1)$$

$$\quad \quad \quad \hookrightarrow x[1] + f(2)$$

$$\quad \quad \quad \quad \quad \hookrightarrow x[2] + f(3)$$

$i = n-1$

$$f(i) = x[i] + f(i+1)$$

$\Rightarrow f(n-1) = x[n-1] + f(n)$

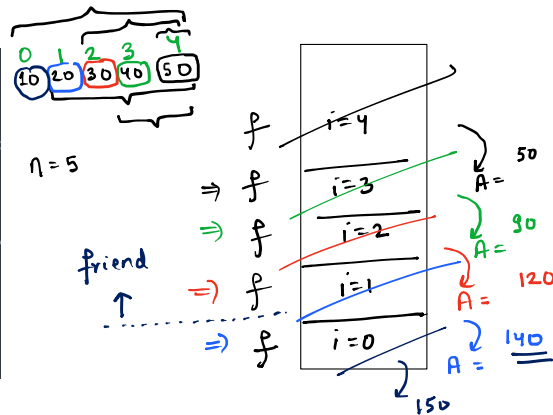
invalid fails

it is a fn that finds the sum of $x[n \dots n-1]$

it is a fn that finds the sum of $x[n-1 \dots n-1] = \{x_{n-1}\}$

the array
 $X[n-1 \dots n-1] = \{X_{n-1}\}$
 $f(n-1) = X_{n-1}$ Base Case

```
int f(int X[], int n, int i) {
    // base case
    if (i == n-1) {
        // f(n-1) = find the sum of
        return X[n-1];
    }
    // recursive case
    // f(i) = find the sum of X[i..n-1]
    // 1. ask your friend to find
    int A = f(X, n, i+1);
    return X[i] + A;
}
```



$f_1 \leftarrow$ Base Case
 f_{n-2}
 f_{n-1}
 $f_n \leftarrow$ orig. problem
 $\eta \cdot \text{const} \Rightarrow \text{time} = O(n)$
 $\eta \cdot \text{const} \Rightarrow \text{space} = O(n)$

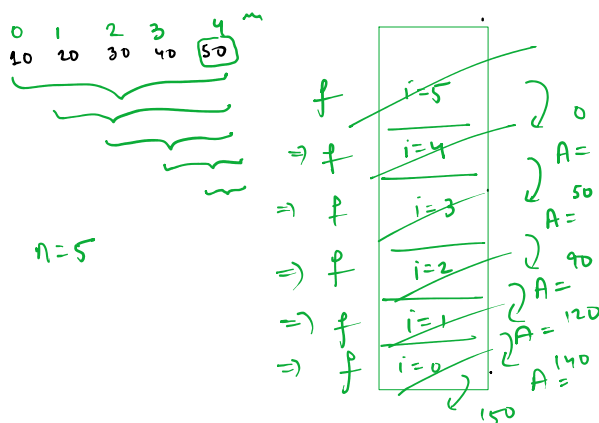
assume $X[n \dots n-1] = \{\}$ exists

$i = n-1$
 $f(n-1) = X[n-1] + \underbrace{f(n)}_{\text{valid}}$ ✓

$i = n$
 $f(n) = \underbrace{X[n]}_{\text{invalid}} + \underbrace{f(n+1)}_{\text{fails}}$

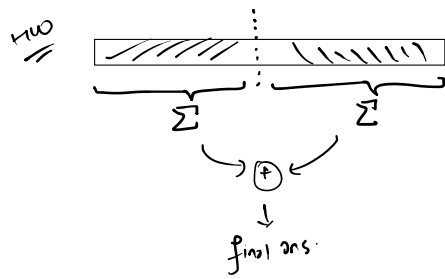
$f(n) = 0$
 it is a fn that finds the sum of $X[n \dots n-1] = \{\}$

```
int f(int X[], int n, int i) {
    // base case
    if (i == n) {
        // f(n) = find the sum
        return 0;
    }
    // recursive case
    // f(i) = find the sum of X[i..n-1]
    // 1. ask your friend to find
    int A = f(X, n, i+1);
    return X[i] + A;
}
```



time: $(\eta+1) \cdot \text{const} \sim O(n)$
 space: $(\eta+1) \cdot \text{const} \sim O(n)$

hw
 10 20 30 40 (50)
 try to solve this
 ques. using
 prefixes



[HW] First Occurrence

First Occurrence

Given an array of **N** integers and a target integer **T**, design a recursive algorithm to find the **index** of its **first occurrence** in the given array.

note : return **-1** if **T** is **not present** in the given array.

Example

Input

T = 20

0	1	2	3	4
10	20	30	20	30

Output : **1**

[HW] Last Occurrence

Last Occurrence

Given an array of **N** integers and a target integer **T**, design a recursive algorithm to find the **index** of its **last occurrence** in the given array.

note : return **-1** if **T** is **not present** in the given array.

Example

Input

T = 20

0	1	2	3	4
10	20	30	20	30

Output : 3

[HW] All Occurrences

All Occurrences

Given an array of **N** integers and a target integer **T**, design a recursive algorithm to find the **indices** of **all** of its **occurrences** in the given array.

note : print **-1** if **T** is **not present** in the given array.

Example

Input

T = 20

0	1	2	3	4
10	20	30	20	30

Output : 1, 3

Is Array Sorted ?

Is Array Sorted ?

Given an array of N integer, design a recursive algorithm to check if it is sorted in the increasing order.

Example

Input

0	1	2	3	4
10	20	30	40	50

Output : True

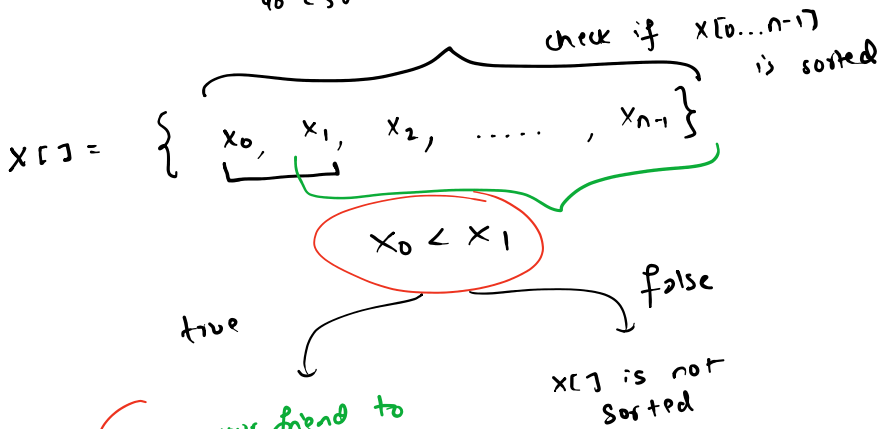
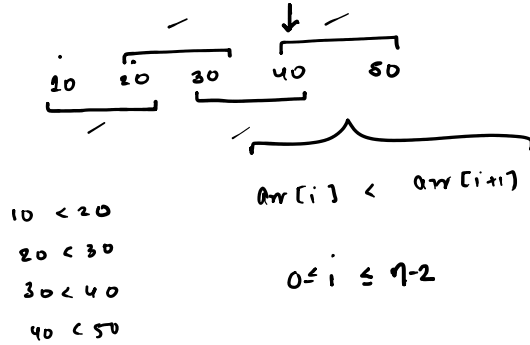
time : $O(n)$

space : $O(1)$

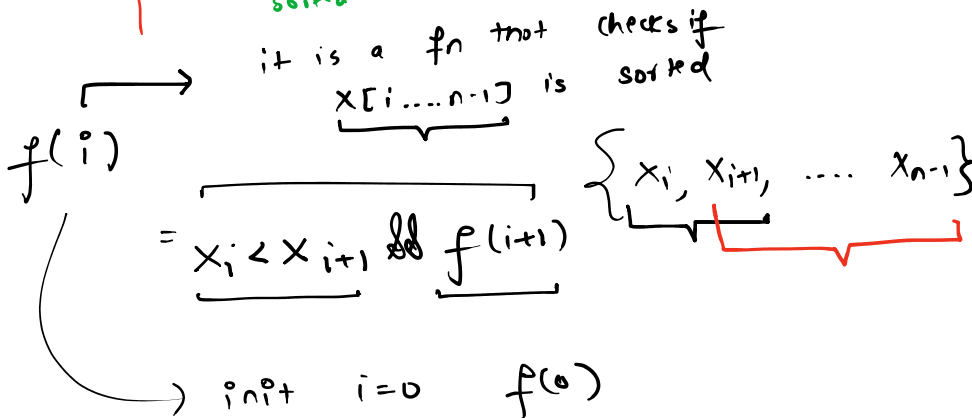
for ($i=0$; $i \leq n-2$; $i++$) {

if ($arr[i] < arr[i+1]$) {
return false;
}

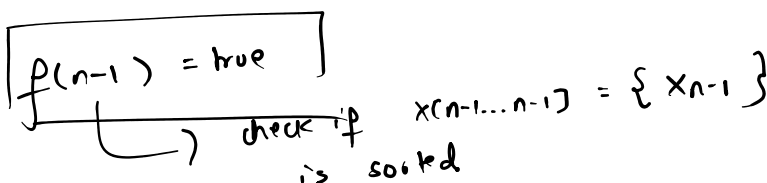
}
return true;



ask your friend to check if $x[1 \dots n-1]$ is sorted



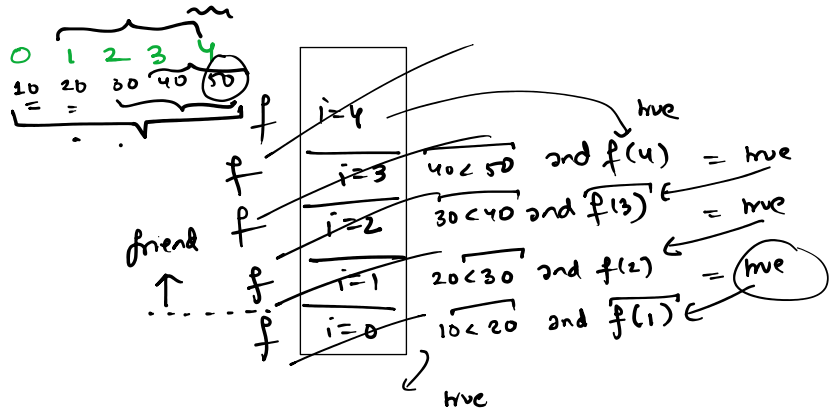
$$f(n-1) = \underbrace{x_{n-1} < (x_n)}_{\text{invalid}} \ \&\& \ f(n) \quad \underline{\underline{\text{fails}}}$$



$n=5$

```
bool f(int X[], int n, int i) {
    // base case
    if (i == n - 1) {
        // f(n-1) = check if X[n-1...n-1] i.e.
        return true;
    }

    // recursive case
    // f(i) = check if X[i...n-1] is sorted
    return X[i] < X[i + 1] and f(X, n, i + 1);
}
```



Time: $O(n)$

Space: $O(n)$

[HW] Binary Search

Binary Search

Given a **sorted** array consisting of **N** distinct integers and a target integer **T**, design a recursive algorithm to find the **index** of its **occurrence** in the array.

note : return **-1** if **T** is **not present** in the given array.

Example

Input

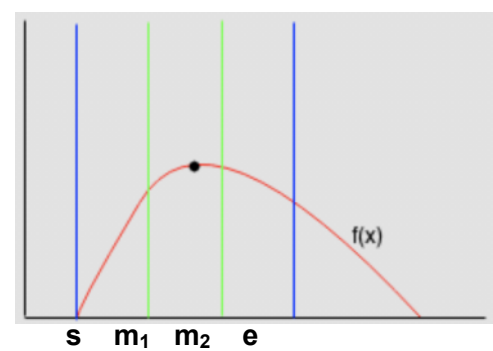
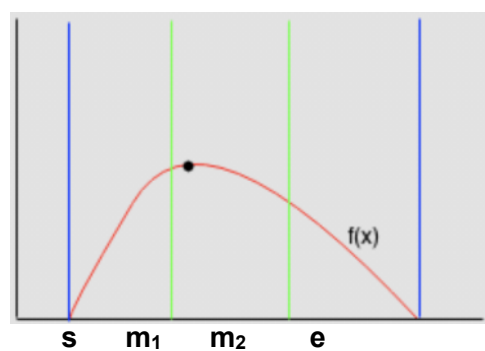
T = 20

0	1	2	3	4
10	20	30	40	50

Output : 1

Ternary Search

Ternary Search is an algorithm that is used to find the **maximum/minimum** value in an **unimodal** function.



Analysis of Ternary Search

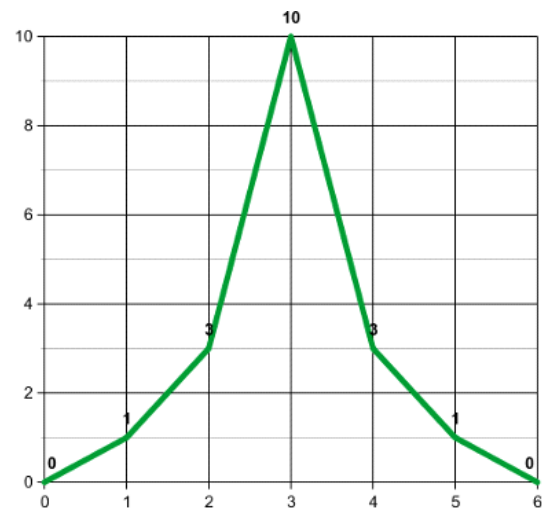
[R][HW] Peak Index in Mountain Array

Peak Index in Mountain Array

Given an unimodal sequence of integers, design an algorithm to find the **maximum** element present in it.

Example

0	1	2	3	4	5	6
0	1	3	10	3	1	0



s **m₁** **m₂** **e**

Merge Sort

⇒ Merge Sort

time: $O(n \log n)$
space: $O(n)$

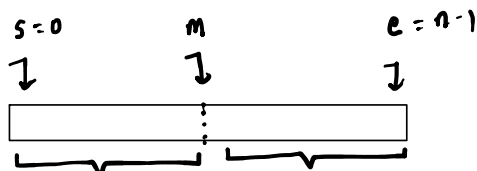
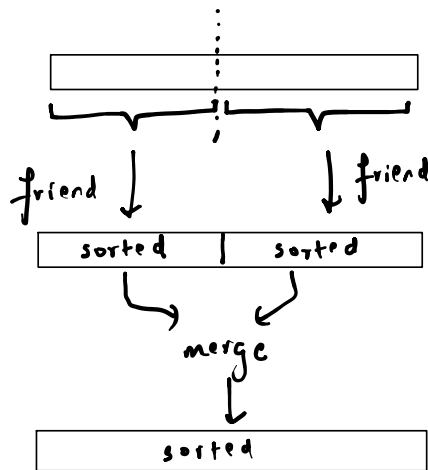
Given an array of **N** integers, sort it using the merge sort algorithm.

Example

0	1	2	3	4
50	40	30	20	10
10	20	30	40	50

The merge sort algorithm operates as per the **divide and conquer** strategy and involves three steps to sort an array

- **divide** the array into two subarray around the **mid-points**
- **recursively** sort the two sub-arrays
- **merge** the two sorted subarrays



$$m = \frac{s+e}{2}$$

$$m = s + \frac{(e-s)}{2}$$

$$f(s, e) \{$$

$arr[s \dots m]$

$arr[m+1 \dots e]$

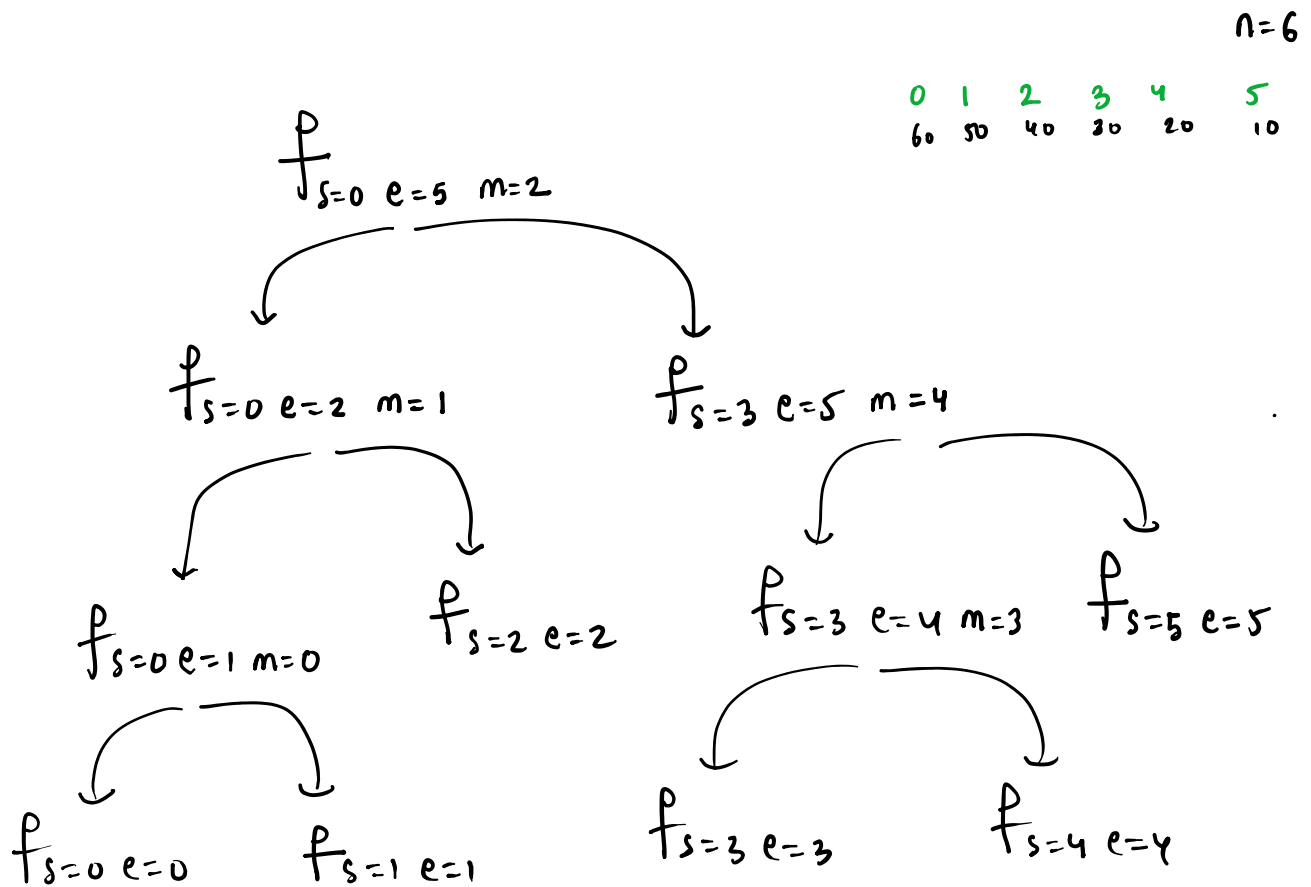
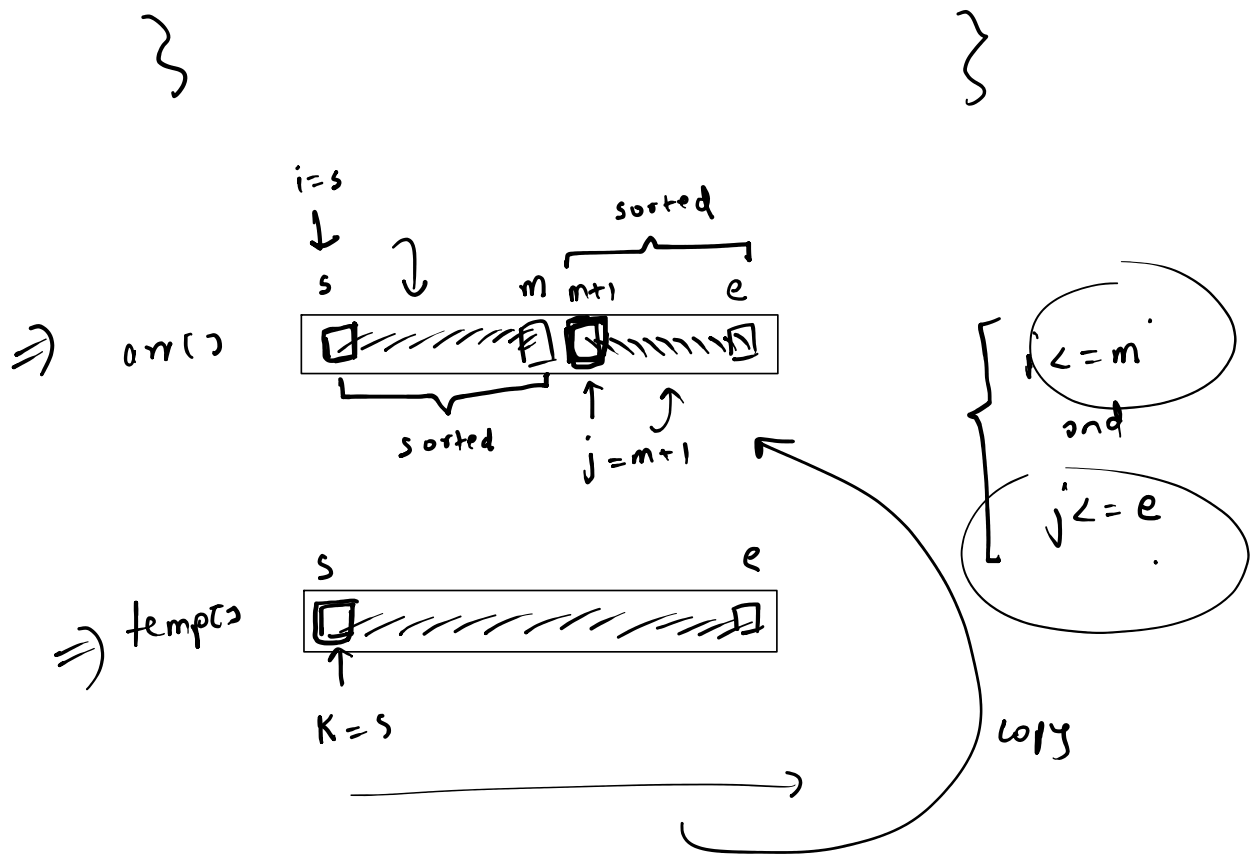
$$f(0, n-1)$$

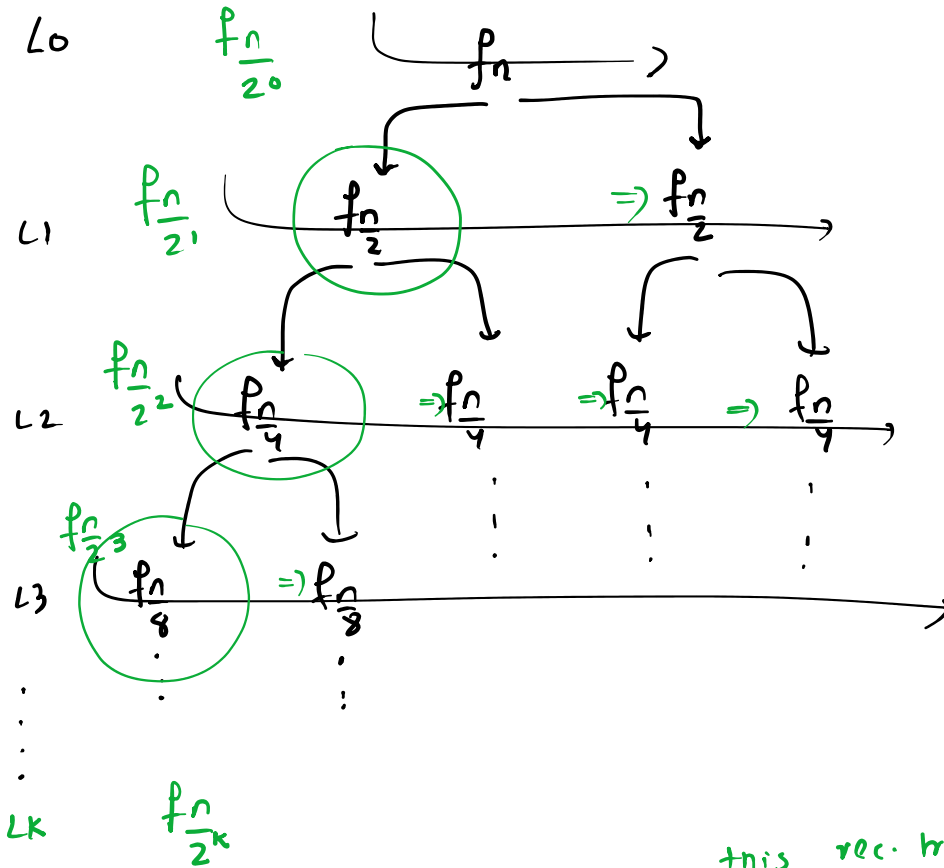
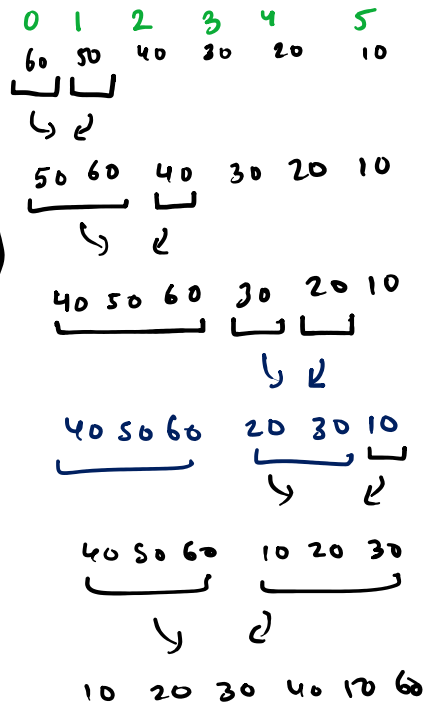
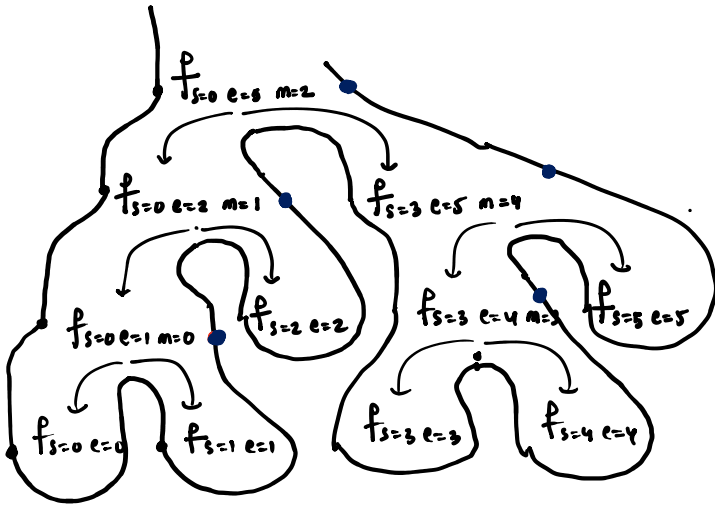
$if (s == e) \{$

$return;$

recursive
case

$m = s + \frac{(e-s)}{2};$
 $f(s, m);$
 $f(m+1, e);$
 $merge(s, m, e)$





$$\frac{n}{2^k} = 1 \Rightarrow 2^k = n$$

$$k = \log_2 n$$

$$\text{tree hg} = \log_2 n$$

this rec. tree has
 $k+1$ levels
 (L_0 to L_k)
 $\therefore \log n + 1$

$$\sim \log n$$

$$\begin{array}{c}
 \overbrace{\log n \cdot \text{const}} \\
 \Downarrow \\
 \# \text{ stk frames}
 \end{array}
 +
 \begin{array}{c}
 \text{merge stk} \\
 \text{frame} \\
 \underbrace{\hspace{1cm}} \\
 n
 \end{array}$$

$\Downarrow$ space for each stk frame

space:

$$\cancel{\log n \cdot c} + n \sim \underline{\underline{O(n)}}$$

[HW] Reverse Pairs

<https://leetcode.com/problems/reverse-pairs/>

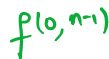
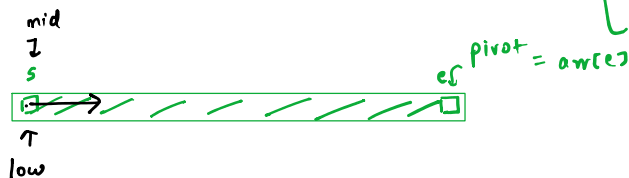
Quick Sort

Given an array of **N** integers, sort it using the quick sort algorithm.

0	1	2	3	4	5
60	50	20	10	40	30
10	20	30	40	50	60

- partition the array into two subarray around the **pivot** such that
 - all the elements in the left partition are $<$ the pivot
 - all the element in the right partition are $>$ the pivot
- recursively** sort the two sub-arrays

usually last el. is taken as pivot

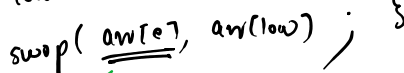

$$\begin{cases} \text{pid}x = \text{partition}(arr, s, e); \\ f(s, \underline{\text{pid}x-1}) \\ f(\underline{\text{pid}x+1}, e) \end{cases}$$


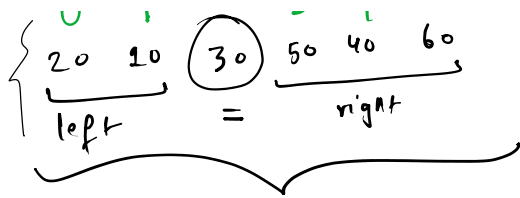
low = 5

```

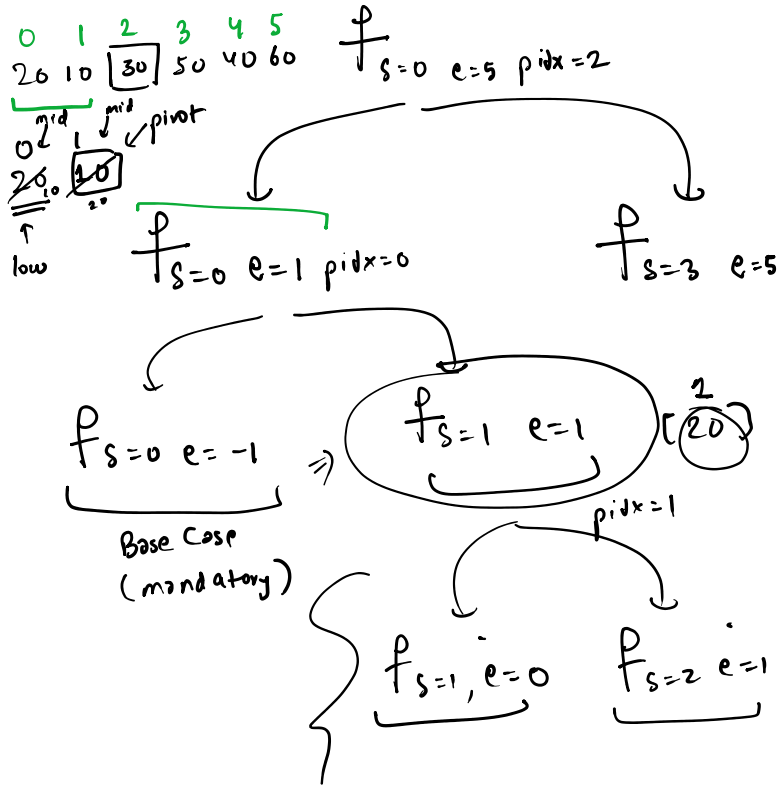
    if (arr[mid] < pivot) {
        swap(arr[mid], arr[low]);
        low++;
    } else {
        mid++;
    }
}

```





0	1	2	3	4	5
60	50	20	10	40	30



Quick Select

Given an array of **N** integers, design an algorithm to find the **Kth smallest** element in it.

Example

K = 3

	0	1	2	3	4	5
arr []	2	7	5	3	8	4

